SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A. Satya Manoj	**Reg. No.:**	192425370
Section / Batch:	2024	**Date:**	

Code of Conduct

I A. Satya Manoj (192425370) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *A. Satya Manoj*

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Code:

```
corpus = [
    [("The","DT"),("boy","NN"),("eats","VBZ"),("rice","NN")],
    [("The","DT"),("girl","NN"),("drinks","VBZ"),("milk","NN")],
    [("A","DT"),("cat","NN"),("drinks","VBZ"),("milk","NN")],
    [("The","DT"),("dog","NN"),("chases","VBZ"),("cat","NN")],
    [("A","DT"),("teacher","NN"),("teaches","VBZ"),("students","NNS")],
    [("Students","NNS"),("study","VBP"),("English","NN")],
    [("Birds","NNS"),("fly","VBP"),("high","RB")],
    [("Children","NNS"),("play","VBP"),("games","NNS")]
]

emission_count = {}
transition_count = {}
tag_count = {}
start_count = {}

for sentence in corpus:
    previous = None

    for i, (word, tag) in enumerate(sentence):

        if tag not in tag_count:
            tag_count[tag] = 0
```

```
        tag_count[tag] += 1

        if tag not in emission_count:
            emission_count[tag] = {}

        if word not in emission_count[tag]:
            emission_count[tag][word] = 0

        emission_count[tag][word] += 1

        if i == 0:
            if tag not in start_count:
                start_count[tag] = 0
            start_count[tag] += 1

        if previous is not None:

            if previous not in transition_count:
                transition_count[previous] = {}

            if tag not in transition_count[previous]:
                transition_count[previous][tag] = 0

            transition_count[previous][tag] += 1

        previous = tag

print("\nEmission Probability\n")

emission_prob = {}

for tag in emission_count:

    emission_prob[tag] = {}

    for word in emission_count[tag]:

        emission_prob[tag][word] = emission_count[tag][word] / tag_count[tag]

        print(tag, "->", word, "=", round(emission_prob[tag][word],3))

print("\nTransition Probability\n")

transition_prob = {}

for tag in transition_count:

    transition_prob[tag] = {}

    total = sum(transition_count[tag].values())

    for next_tag in transition_count[tag]:

        transition_prob[tag][next_tag] = transition_count[tag][next_tag] / total

        print(tag, "->", next_tag, "=", round(transition_prob[tag][next_tag],3))

print("\nStart Probability\n")

start_prob = {}

total_sentences = len(corpus)

for tag in start_count:
```

```
    start_prob[tag] = start_count[tag] / total_sentences

    print(tag, "=", round(start_prob[tag],3))

states = list(tag_count.keys())

sentence = ["The","cat","drinks","milk"]

V = [{}]
path = {}

for state in states:

    start = start_prob.get(state,0)
    emit = emission_prob.get(state,{}).get(sentence[0],0)

    V[0][state] = start * emit
    path[state] = [state]

for t in range(1,len(sentence)):

    V.append({})
    new_path = {}

    for current in states:

        emit = emission_prob.get(current,{}).get(sentence[t],0)

        best_prob = -1
        best_state = None

        for previous in states:

            trans = transition_prob.get(previous,{}).get(current,0)

            prob = V[t-1][previous] * trans * emit

            if prob > best_prob:

                best_prob = prob
                best_state = previous

        V[t][current] = best_prob

        if best_state is not None:
            new_path[current] = path[best_state] + [current]
        else:
            new_path[current] = [current]

    path = new_path

best_prob = -1
best_state = None

for state in states:

    if V[-1][state] > best_prob:

        best_prob = V[-1][state]
        best_state = state

print("\nPredicted POS Tags\n")
```

```
print("Sentence :", " ".join(sentence))
print("Tags     :", " ".join(path[best_state]))
```

Output Clear

```
Emission Probability

DT -> The = 0.6
DT -> A = 0.4
NN -> boy = 0.1
NN -> rice = 0.1
NN -> girl = 0.1
NN -> milk = 0.2
NN -> cat = 0.2
NN -> dog = 0.1
NN -> teacher = 0.1
NN -> English = 0.1
VBZ -> eats = 0.2
VBZ -> drinks = 0.4
VBZ -> chases = 0.2
VBZ -> teaches = 0.2
NNS -> students = 0.2
NNS -> Students = 0.2
NNS -> Birds = 0.2
NNS -> Children = 0.2
NNS -> games = 0.2
VBP -> study = 0.333
VBP -> fly = 0.333
VBP -> play = 0.333
RB -> high = 1.0
Transition Probability

DT -> NN = 1.0
NN -> VBZ = 1.0
VBZ -> NN = 0.8
VBZ -> NNS = 0.2
NNS -> VBP = 1.0
VBP -> NN = 0.333
VBP -> RB = 0.333
VBP -> NNS = 0.333

Start Probability

DT = 0.625
NNS = 0.375

Predicted POS Tags

Sentence : The cat drinks milk
Tags     : DT NN VBZ NN

=== Code Execution Successful ===
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	4	3	3	3	3		80
2	4	3	3	3	3		
3	4	3	3	3	3		
4	4	3	3	3	3		
5	4	3	3	3	3		

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____